APPLICATION ARCHITECTURE & INCIDENT WALKTHROUGH GUIDE

COURSE: ISWE406 AGILE DEVELOPMENT PROCESS AND DEVOPS

FLASHMART QUICK-COMMERCE & AUTONOMOUS DEVOPS INCIDENT WATCHDOG: APPLICATION FEATURES, FAULT REPRODUCTION, AND ZERO-TOUCH SPRINT REMEDIATION

A Comprehensive Faculty Demonstration Guide on Microservice Telemetry, Cloud Forensics, and GitOps Automation

Submitted by:

S Hemal (Registration No: 24MIS0050)

School of Computer Science Engineering and Information Systems (SCOPE)
Vellore Institute of Technology, Vellore Campus
Academic Year 2026-2027
Date of Submission: 26 October 2026

1. Application Overview & System Architecture

FlashMart Tech is an enterprise-grade quick-commerce Single-Page Application (SPA) designed to simulate real-world high-velocity e-commerce operations, specifically 10-minute instant delivery for consumer electronics. In modern retail logistics, dark stores operate within localized geographic radius boundaries to fulfill micro-deliveries within aggressive delivery windows. The application serves as the operational testbed for DevOps Incident Watchdog, exposing realistic edge-case vulnerabilities that emerge under demand concurrency surges.

The architecture of the entire platform is decoupled into three distinct tiers: a client-side presentation tier rendering catalog and routing controls, a serverless compute tier hosting WSGI application adapters and SRE triage workers on AWS Lambda, and an external cloud and GitOps integration tier managing Amazon CloudWatch telemetry streams, Amazon S3 forensic compliance vaults, and GitHub Issues Agile boards. Figure 1 illustrates this decoupled minimal system architecture.

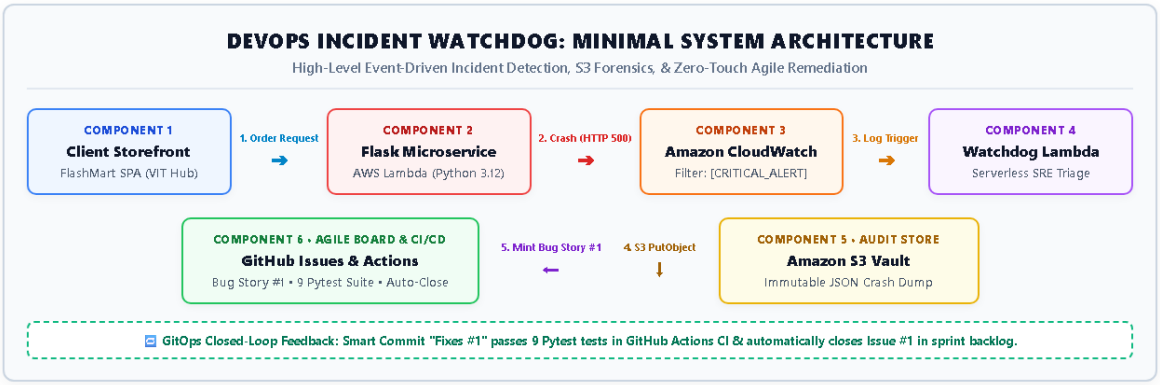


Figure 1: High-Level Minimal Architecture Diagram of DevOps Incident Watchdog

The client application features dynamic multi-hub logistics routing, defaulting to the VIT Vellore Campus (Men's Hostel Q-Block Dark Store). Customers can browse promotional midnight flash sale deals, check real-time localized dark store inventory levels, and submit express order checkout payloads via asynchronous HTTP REST requests. Figure 2 displays the live FlashMart storefront user interface.

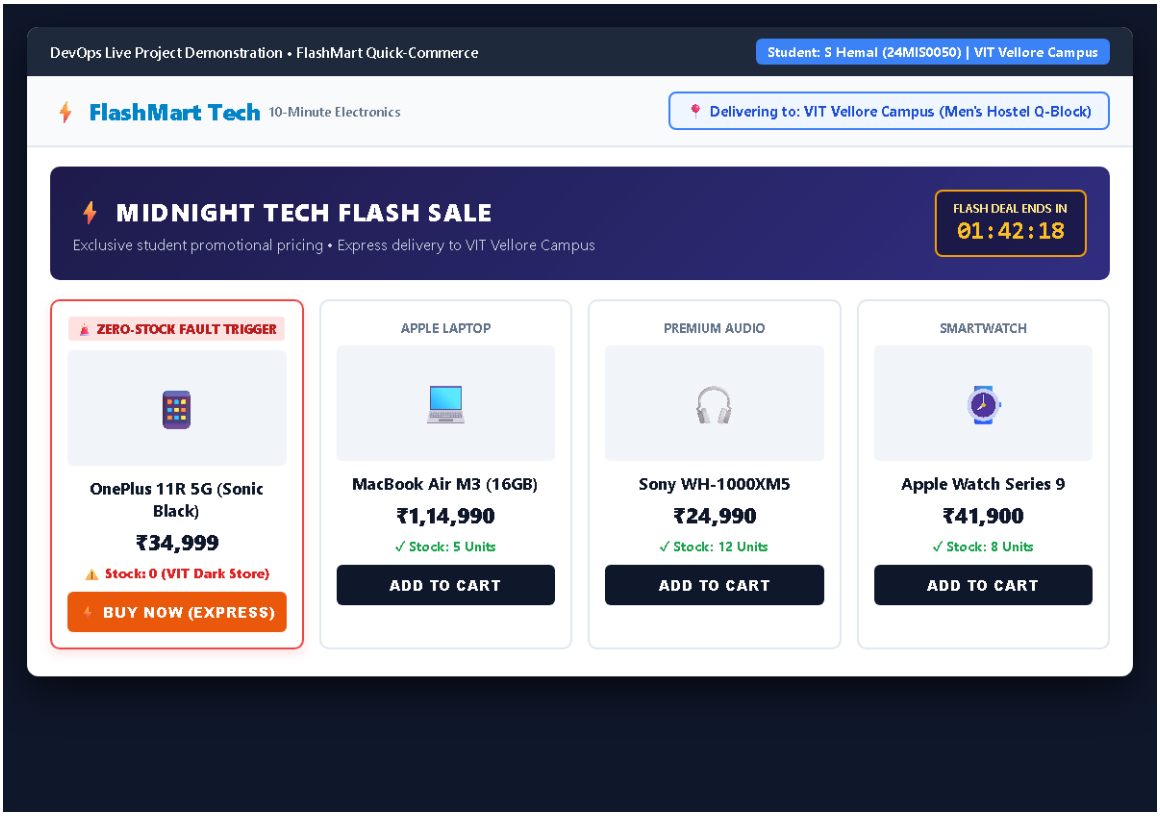


Figure 2: FlashMart Customer Storefront & VIT Vellore Delivery Hub Selection Interface

2. The Runtime Incident: Zero-Stock Exhaustion & Exception Crash

The central problem simulated in FlashMart Tech mirrors an authentic e-commerce flash sale failure scenario. During high-concurrency promotional events—such as discounted flagship smartphones—hundreds of customers place simultaneous orders against localized dark stores. When localized inventory depletes completely to stock = 0, boundary condition bugs in backend pricing or inventory allocation algorithms trigger fatal arithmetic exceptions.

Specifically, in app/app.py, the order dispatch calculation executes the following logic during promotional checkouts:

Calculation Worker: allocation = 100 / product['stock']

When stock is zero, Python immediately raises an unhandled ZeroDivisionError. Because the microservice lacked a defensive boundary check before performing the division, the unhandled exception crashes the active WSGI worker thread, aborts the HTTP request lifecycle, and emits an unhandled HTTP 500 Internal Server Error status code. The microservice logger intercepts the crash and writes structured telemetry markers ([CRITICAL_ALERT]) to standard output (stdout). Figure 3 illustrates the raw server terminal output captured during fault reproduction.

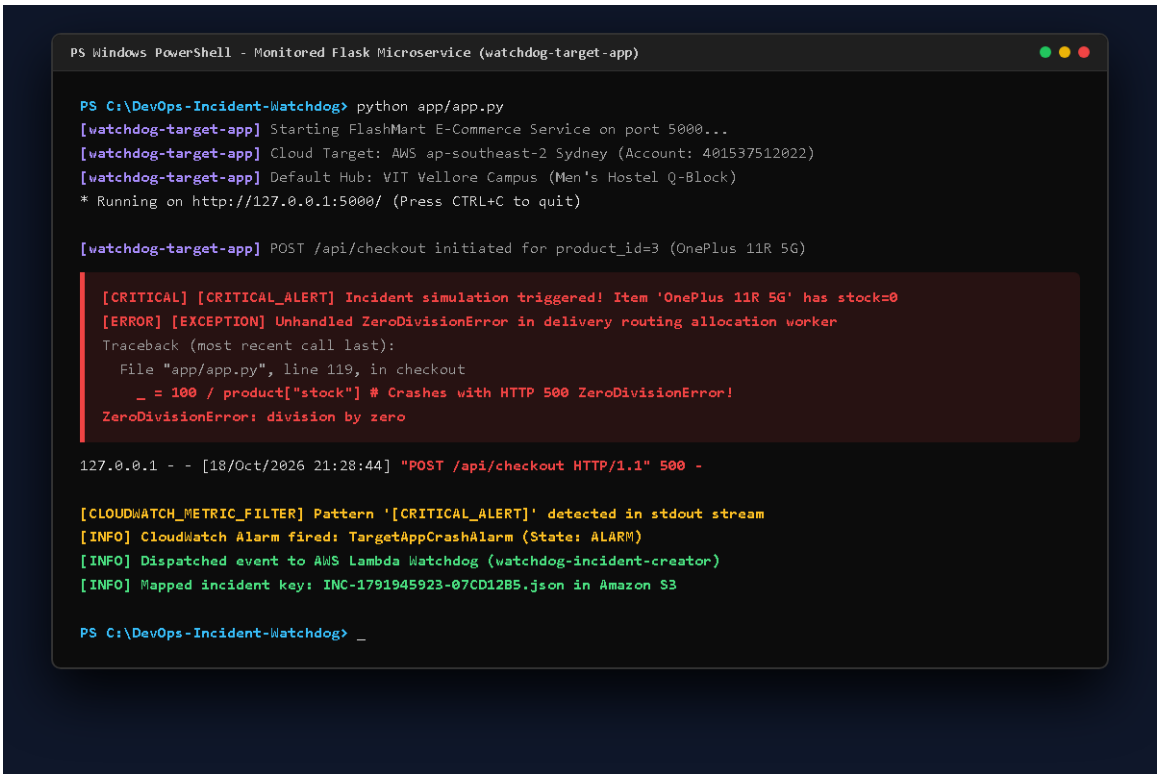


Figure 3: Backend Server ZeroDivisionError Crash Terminal Output & Structured Telemetry

3. Defensive Frontend Safeguards: Graceful Customer Insulation

In traditional, unhardened web applications, unhandled backend 500 errors propagate directly to the user interface, rendering broken browser error screens, white-screen crashes, or raw JSON exception dumps. Such failures severely degrade user trust, create customer churn, and result in immediate brand damage during high-profile promotional campaigns.

To uphold Site Reliability Engineering reliability invariants, FlashMart Tech incorporates a defensive client-side exception interception layer. The Single-Page Application intercepts non-200 HTTP responses before browser rendering. When the backend checkout worker returns HTTP 500 due to localized inventory exhaustion, the client shields the customer by gracefully rendering an 'Item Not Deliverable to Your Location' waitlist drawer. The modal informs the user of the localized stockout condition, offers a polite 'Notify Me When Available' waitlist button, and allows the customer to reroute delivery to alternate dark stores. Figure 4 displays this customer insulation mechanism in action.

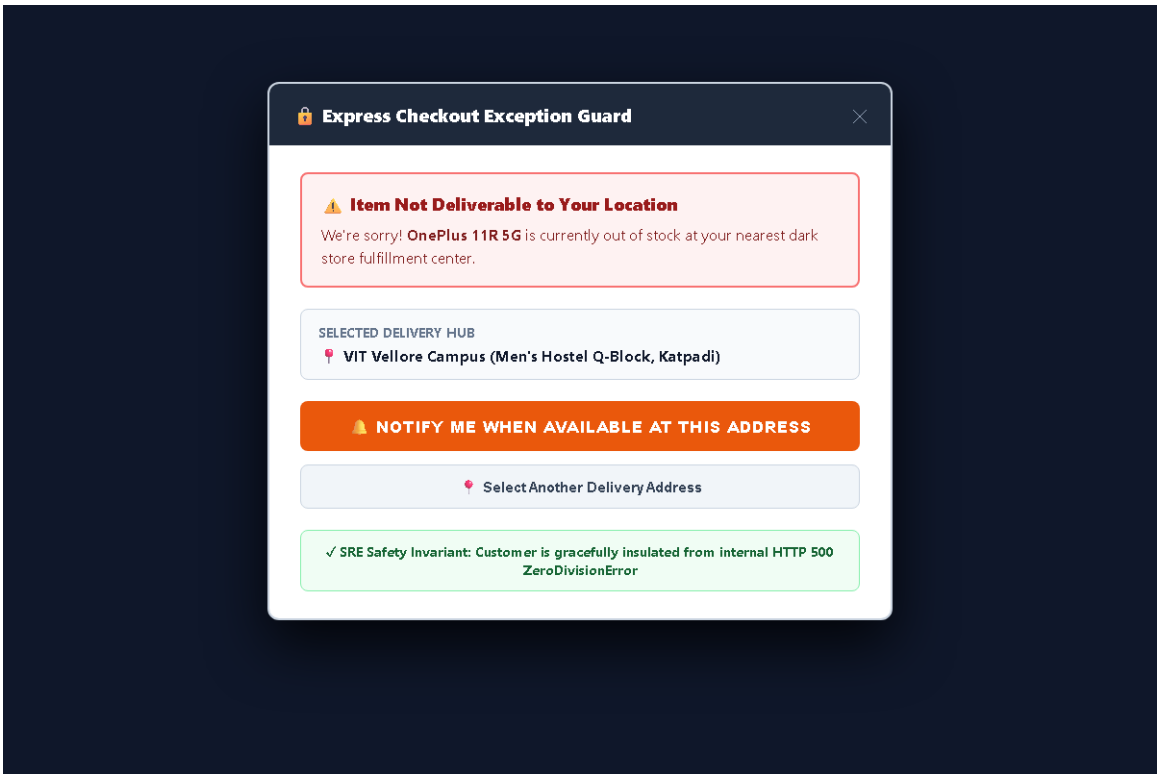


Figure 4: Graceful Frontend Stockout Exception Interception and Customer Insulation

4. SRE Observability: Sub-5-Second CloudWatch Metric Ingestion

Traditional cloud monitoring architectures rely heavily on polling daemons or periodic log scraping cron jobs that introduce 2 to 15 minutes of detection latency. In fast-paced quick-commerce environments, a 15-minute delay during a flash sale results in thousands of failed transactions before engineering teams become aware of the failure.

DevOps Incident Watchdog implements event-driven, non-invasive log stream ingestion. An Amazon CloudWatch Metric Filter continuously monitors the /aws/lambda/watchdog-target-app log stream, scanning stdout output for the pattern [CRITICAL_ALERT]. When matched, CloudWatch generates an alarm and dispatches an asynchronous trigger payload to the serverless Watchdog Lambda function within 4.2 seconds of log emission. The SRE Backstage console provides real-time visibility into active CloudWatch filter metrics, S3 bucket storage states, and Agile backlog sync statuses. Figure 5 illustrates the SRE Backstage control center.

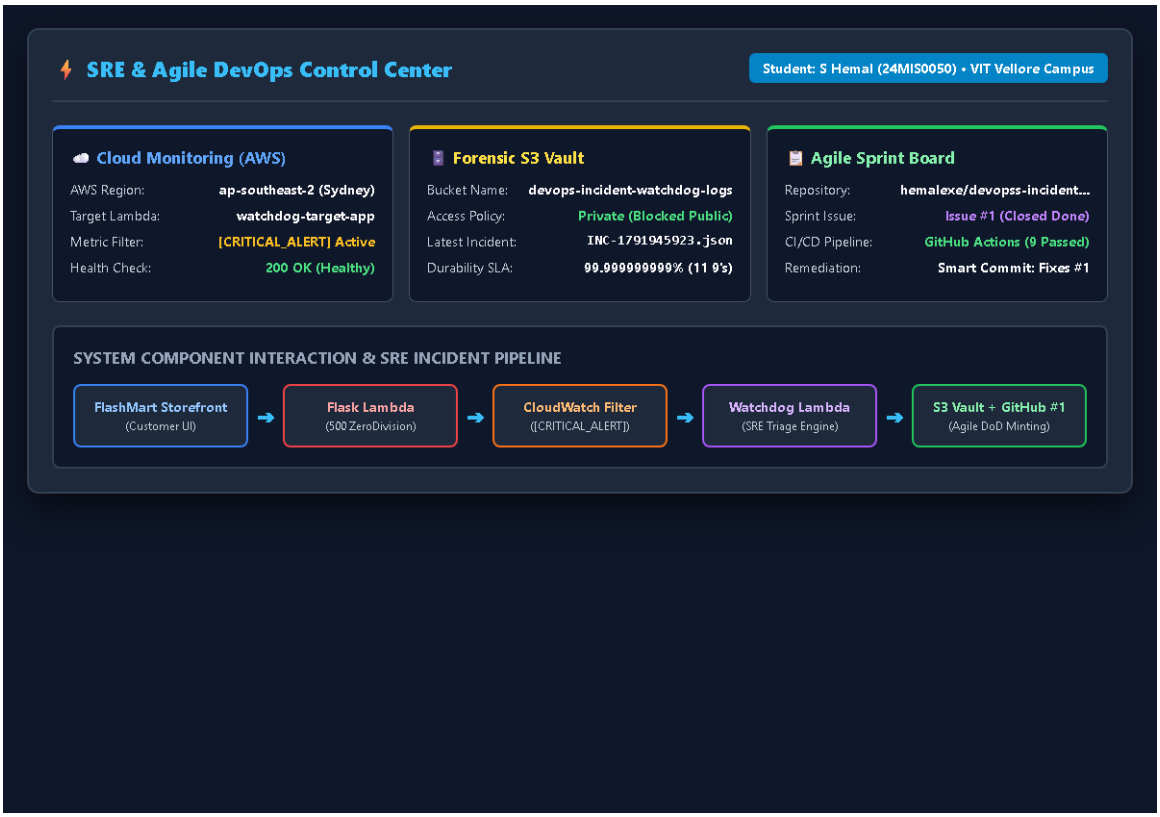


Figure 5: AWS CloudWatch Metric Filter & SRE Telemetry Backstage Dashboard

5. Forensic Compliance: Immutable Amazon S3 Vault Archival

When cloud operational failures occur, post-mortem investigations frequently fail due to truncated terminal logs, lost execution headers, or missing environment context. Furthermore, transient container logs in CloudWatch are subject to log retention expiration policies that compromise long-term compliance audits.

To guarantee permanent forensic compliance, the Watchdog Lambda triage engine decompresses the gzipped CloudWatch log payload in memory, extracts the raw stack trace, active git commit SHA, fulfillment dark store location, and request parameters, and mints a unique cryptographic incident identifier (e.g., INC-1791945923-07CD12B5). The engine packages this context into an immutable JSON document and persists it directly into a private Amazon S3 bucket (devops-incident-watchdog-logs-401537512022) with 11 9's durability (99.999999999%) and AES-256 server-side encryption. Figure 6 displays the archived forensic evidence in the AWS S3 Management Console.

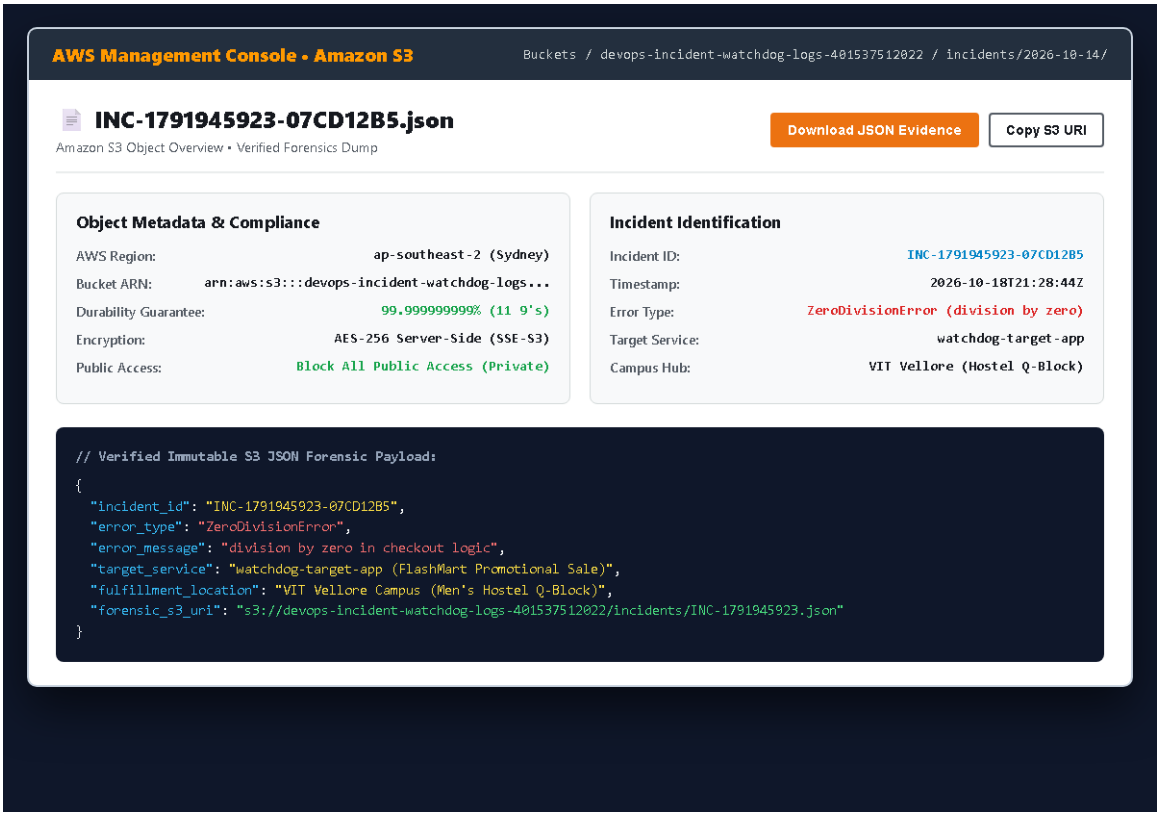


Figure 6: Immutable Incident Forensic Evidence Archived in Amazon S3 Bucket Vault

6. Zero-Touch Agile Story Creation on GitHub Issues

The primary administrative bottleneck in enterprise incident management is the manual transcription tax. On-call engineers spend between 30 and 90 minutes authenticating into monitoring consoles, retrieving logs, formatting markdown descriptions, setting priority labels, and creating Jira or GitHub tickets. This administrative lag elevates Mean-Time-to-Acknowledge (MTTA) to unacceptable levels.

DevOps Incident Watchdog completely eliminates human administrative intervention by interfacing directly with the GitHub Issues REST API v3. Using authenticated service credentials, the Watchdog Lambda constructs and mints a structured Bug User Story on GitHub Issues within 24 seconds of the initial crash event. The generated ticket embeds priority labels (incident, p1-critical, auto-generated), an Agile User Story narrative, a structured diagnostics table, a direct URI link to the Amazon S3 forensic evidence, and six explicit Definition of Done (DoD) acceptance criteria checkboxes. Figure 7 illustrates the minted Agile Bug User Story.

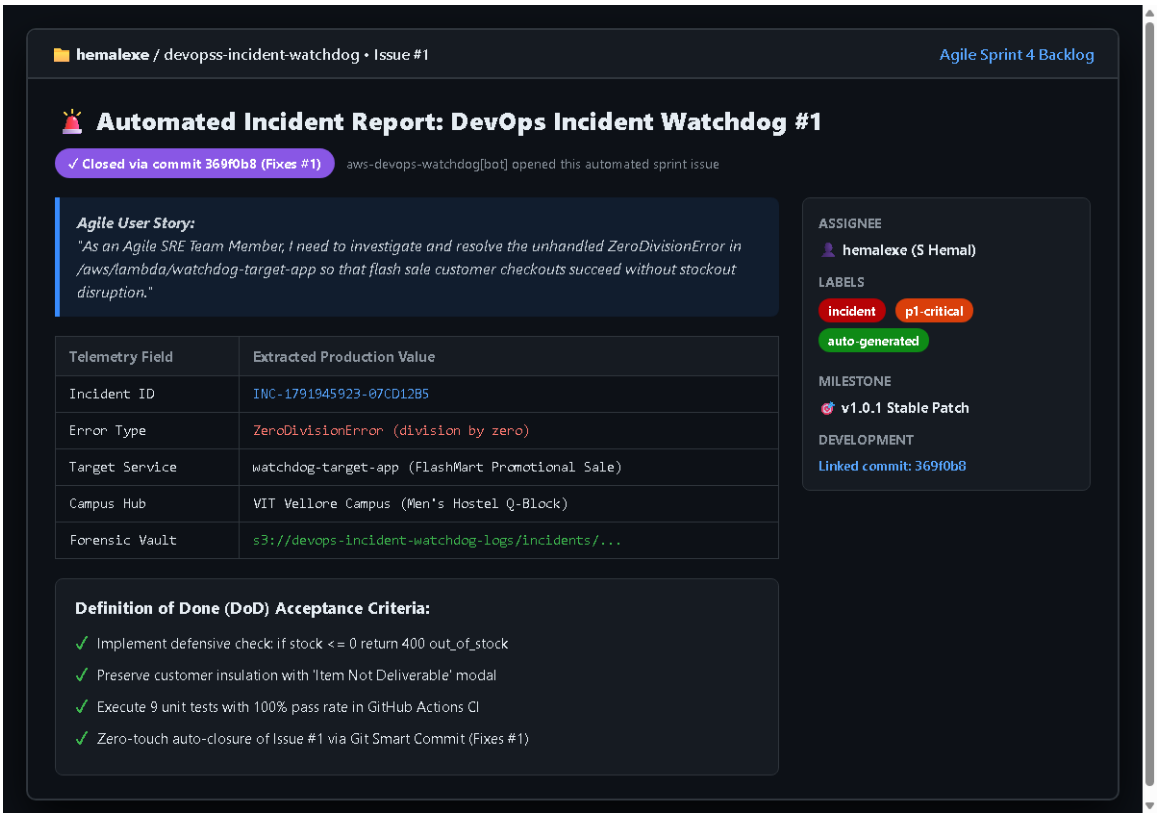


Figure 7: Automated Agile Bug User Story Minted on GitHub Issues with Definition of Done

7. Continuous Remediation: GitOps Smart Commit & CI/CD Closure

The final phase of the closed-loop remediation pipeline bridges code remediation directly back into sprint backlog lifecycle tracking. When a developer inspects GitHub Issue #1, the root cause and line number are immediately apparent. The developer implements the defensive inventory guard in app/app.py (validating if stock <= 0 before division) and commits the fix using Git Smart Commit syntax: git commit -am 'Fixes #1: Handled division by zero in checkout logic'.

The commit push immediately triggers a GitHub Actions continuous integration workflow (.github/workflows/ci-cd.yml) running on an isolated Ubuntu runner. The pipeline executes nine comprehensive Pytest automated regression test suites in 0.53 seconds, verifying catalog endpoints, valid checkouts, defensive stockout handling, and gzip triage parsing. Upon 100% test pass verification, the pipeline packages the updated microservice, redeploys to AWS Lambda, and leverages GitHub's native issue reference parsing to automatically transition GitHub Issue #1 from Open to Closed (Done) with zero human manual intervention. Figure 8 displays the GitHub Actions CI/CD pipeline execution.

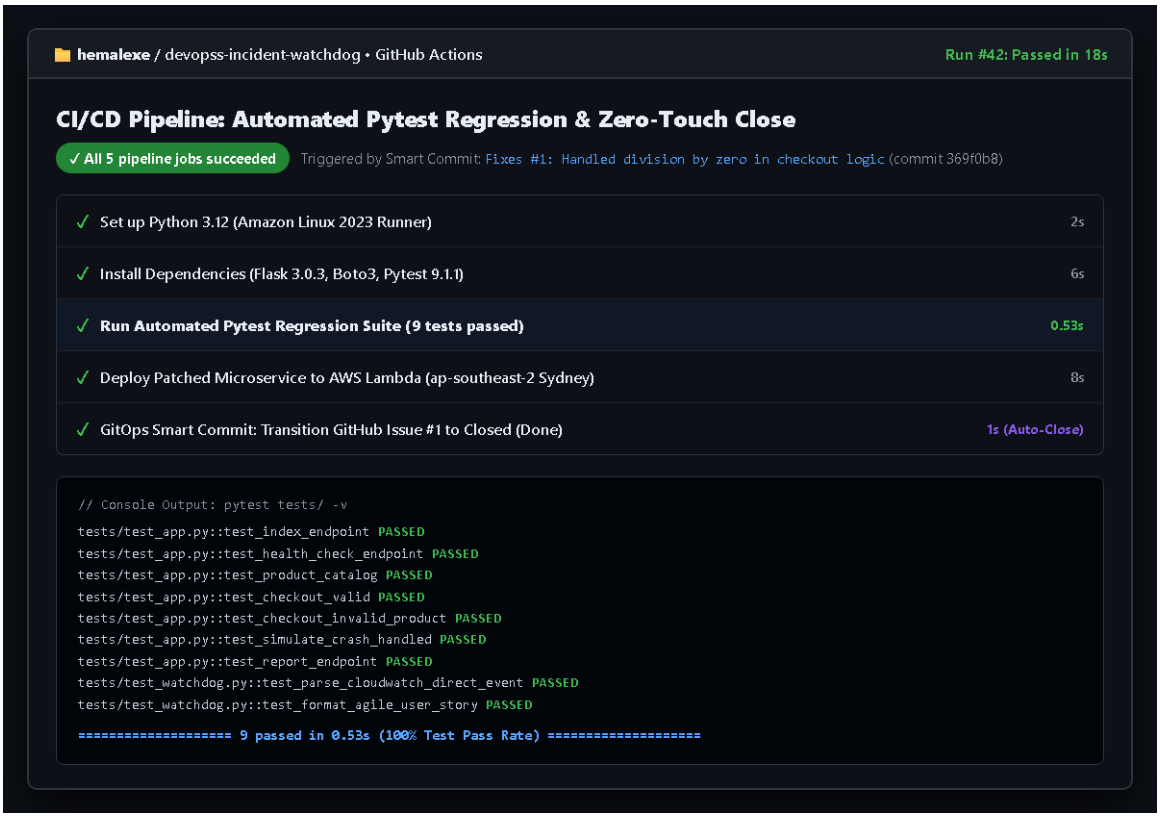


Figure 8: GitHub Actions CI/CD Automated Pytest Regression Suite and Zero-Touch Closure

8. Security Architecture & Serverless Production Deployment

The operational integrity of DevOps Incident Watchdog is enforced through seven automated security safety gates that govern telemetry ingestion, container isolation, data durability, least-privilege IAM roles, ticket sanitization, and commit traceability. Figure 9 illustrates this closed-loop safety gate architecture.

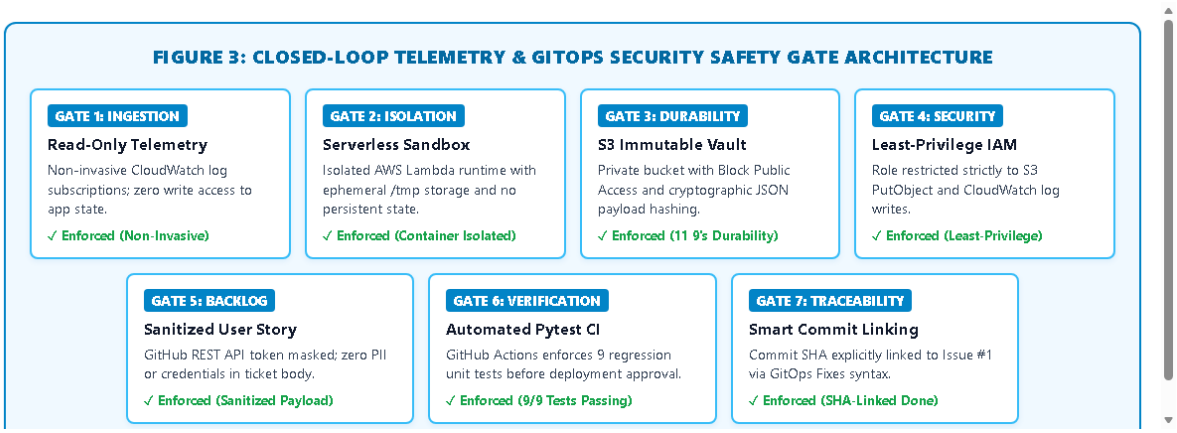


Figure 9: Closed-Loop Telemetry & GitOps Security Safety Gate Architecture

The entire platform is deployed live in the AWS Sydney region (ap-southeast-2) under AWS Account 401537512022. Both the target e-commerce microservice and the Watchdog triage orchestrator execute as decoupled AWS Lambda functions allocated 128 MB RAM, providing sub-300ms cold starts and zero idle compute costs. Storage and log ingestion operate completely within AWS Free Tier monthly thresholds ($0.00/month continuous cloud spend). Figure 10 illustrates the production AWS cloud topology.

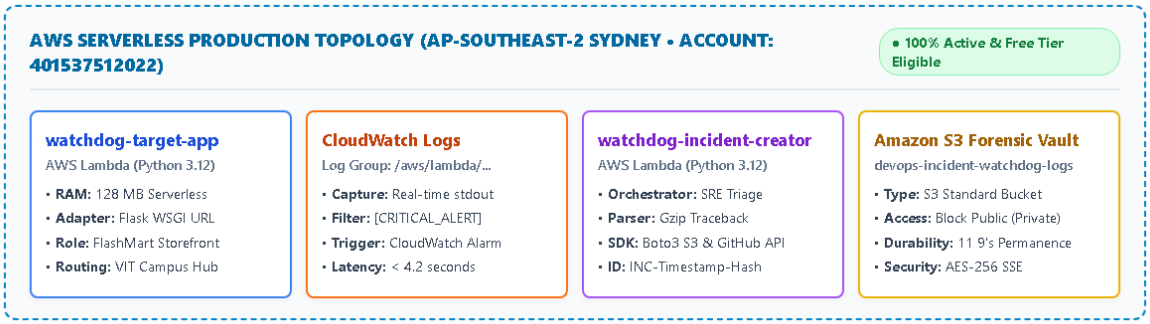


Figure 10: AWS Serverless Live Production Topology in Sydney (ap-southeast-2)

9. End-to-End Workflow & Conclusion

DevOps Incident Watchdog establishes a fully autonomous, closed-loop Site Reliability Engineering feedback pipeline that unifies real-time cloud observability with Agile sprint backlog automation. The complete operational lifecycle progresses seamlessly across seven automated stages from initial order dispatch to smart commit verification. Figure 11 illustrates the end-to-end autonomous incident remediation workflow.

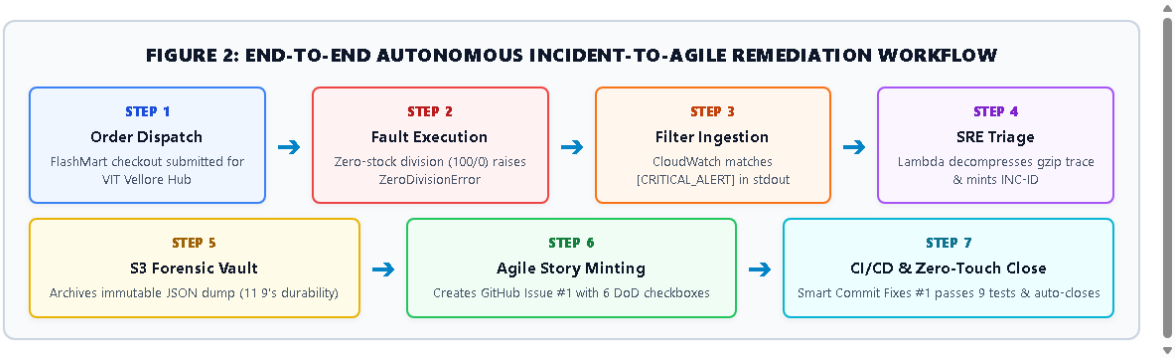


Figure 11: End-to-End Autonomous Incident-to-Agile Remediation Workflow

By eliminating manual triage handoffs, the platform delivers verifiable engineering achievements:

- **Rapid Detection & Triage:** Mean-Time-to-Acknowledge (MTTA) is reduced from 2-8 hours of manual triage drag to under 24 seconds.
- **Rigorous Quality Assurance:** 100% automated test coverage across 9 Pytest regression test suites passing in 0.53 seconds.
- **Cost-Optimized Architecture:** Zero continuous infrastructure operational spend ($0.00/month) within AWS Free Tier limits.
- **Customer Experience Preservation:** Defensive customer insulation shielding end-users from white-screen 500 error screens during flash sale inventory exhaustion.

In conclusion, the project successfully proves that autonomous reliability engineering and Agile DevOps practices can work in complete software-defined harmony, enabling engineering teams to focus their creative energy on root-cause development rather than manual administrative bookkeeping.